Sentinel SOC — AI Strategy Council Report v4

Fourth adversarial review · 22 July 2026 Three seats, three vendors, three rounds, 32.8 minutes, votes recorded verbatim.

The finding, stated first

The fourth council voted **FAILURE_RELOCATED_AGAIN: YES**, unanimously, for the fourth consecutive time.

The V4 remediation was the largest so far. It wired PostgreSQL for real and verified it against a live server, replaced an in-memory queue with a durable one, added leader election, deleted dead infrastructure, and replaced a 22-case corpus with 7,075 third-party cases. Every one of those things is real and independently checkable.

It also introduced **seven new defects, every one an instance of a class the council had already named.** The sharpest is worth stating plainly, because it is the whole pattern in miniature:

> *The remediation deleted a table to end split-authority, left a reference to that table inside* `apply_retention`*, then* ***scheduled*** *that function via its new leader election — and the new* `PeriodicJob` *caught the resulting exception into a field nobody reads. The maintenance job V4 was proudest of activating failed* ***100% of runs, silently.*** *When that was fixed, it stopped crashing and immediately* ***deadlocked instead*** *— because* `elected()` *held a non-reentrant lock across its* `yield`*, and the now-working job asked for the same lock. Repairing one defect activated another.*

All seven findings were verified against running code and have been fixed. **The votes are not revised.** The council voted on the code as it stood.

How this review was conducted

Seat	Vendor	Model	Round timings
Claude Opus 4.8	Anthropic	`claude-opus-4-8`	452s / 280s / 104s
Codex GPT-5.6-sol	OpenAI	`gpt-5.6-sol`	411s / 188s / 163s
Antigravity (Gemini)	Google	`agentapi --model=pro`	808s / 995s / 75s

Three rounds — opening (independent), rebuttal (each seat reads the others), closing (votes and plan). Nine transcripts, 84,416 characters, preserved verbatim in `council/transcript/`.

A fourth seat, Gemini 2.5 Flash, remains **unavailable**: Google terminated free-tier CLI access for personal accounts. Recorded rather than quietly dropped.

A process failure worth recording

The first launch of this run **completed in seconds having asked no model anything.** The runner resumes from any existing transcript file, and the v3 transcripts were still on disk, so it reported nine successful seats and

reused the previous review's answers verbatim.

Had that not been checked against the log, this report would have been built from v3 opinions about v4 code. It is recorded here because a review that hides its own near-miss has no standing to report anyone else's.

Conflicts of interest

1. **The Claude seat implemented all four remediations.** It was told so and instructed to be harder on its own work. Its closing states the anti-overclaiming tests it wrote are *"substring theater"* and names three false claims that pass them.
2. **The Antigravity seat is from the vendor whose model wrote the original code.** It found the most severe defect of this round — the deadlock — which the other two seats missed.
3. **This report was assembled by the model occupying the Claude seat.** Mitigation: votes are quoted verbatim and all nine transcripts ship alongside.

The recorded votes

Question	Claude	Codex	Antigravity
Classes actually eliminated	PARTIAL	PARTIAL	PARTIAL
Failure relocated again	**YES**	**YES**	**YES**
Honest about its own state	**NO**	**NO**	**NO**
Safe to run a real pilot	**NO**	**NO**	**NO**
Viable commercial product	**NO**	**NO**	**NO**
Viable venture company	**NO**	**NO**	**NO**
Evidence ledger is a real moat	**NO**	**NO**	**NO**
Regression introduced	**YES**	**YES**	**YES**

Unanimous on all eight, for the fourth review running.

Biggest single risk, verbatim:

> ***Claude:*** *Retention silently deletes attestation artifacts; failure evidence is silently dropped via malformed DLQ.*

> ***Codex:*** *API acknowledges processing before durable work exists.*

> ***Antigravity:*** *Silent work dropping between API response and deferred background queue enqueue.*

Part I — The seven findings

Each was reproduced against running code before being fixed. Findings 1–3 were caught during rounds 1–2; 4–7 in the closing round.

F1 — Scheduled retention crashed on every run, silently *(Critical)*

`apply_retention` still named the `incidents` table that V4 had deleted, so it raised `OperationalError` on every invocation. Because V4 had just *scheduled* it via `leader.PeriodicJob`, and that class caught exceptions into a `last_error` field with no reader, a 100%-failing job was indistinguishable from a healthy one.

Class: silent scheduled failure. Verified by direct invocation.

F2 — A second metrics store survived the split-authority cleanup *(High)*

`app/metrics/system_metrics.py` kept counters in a JSON file, was written on the simulation path, and was never read by `/metrics` — free to diverge from the database permanently. Exactly the class V4 claimed to have eliminated.

Class: split authority. Module deleted.

F3 — Leader election deadlocked on SQLite *(Critical)*

`db._connect()` holds a process-wide **non-reentrant** `threading.Lock` for its whole context. `elected()` opened a connection and yielded inside it; the elected job then called `db._connect()` itself and blocked on a lock its own caller owned. Permanent, silent deadlock of the maintenance thread on the first scheduled run.

Found by the Antigravity seat alone. Reproduced with a 30-second probe that hung.

It was invisible until F1 was fixed. While retention crashed instantly the lock was released instantly too. Repairing one defect activated the other — the pattern demonstrated inside a single session.

Class: lock held across a callback.

F4 — `INSERT OR IGNORE` lost its meaning on PostgreSQL *(High)*

`to_postgres()` stripped `OR IGNORE` and appended nothing. On PostgreSQL a duplicate event therefore raised `IntegrityError` instead of being ignored — **deduplication was broken on the enterprise backend only.** The module docstring advertised the correct `ON CONFLICT DO NOTHING` mapping the entire time.

Worse: a v3 regression test **asserted the buggy output as correct.** That test had been protecting the defect. A regression test encoding wrong behaviour is worse than no test, because it makes the defect look deliberate.

Class: semantics lost in translation.

F5 — Retention violated the invariant the state machine enforces *(Critical)*

`repository.transition()` refuses to enter `COMPLETED` without a report row (`ArtifactMissing`). Retention deleted each table on its **own** timestamp, so an incident whose report aged out before its state row did was left `COMPLETED` **with no report** — precisely the state the state machine exists to make unreachable.

A long-running investigation (report written early, incident updated late) hits this on ordinary data. Reproduced.

Class: an invariant enforced on write but not maintained under deletion. This is the deepest finding of the round: a guard on the write path is not a guarantee unless the delete path preserves it.

F6 — The dead-letter call transposed its arguments *(High)*

`dlq.enqueue(tenant_id, kind, payload, max_attempts)` was called as `(tenant_id, incident_id, "pipeline", str(e))`. So `kind` held the incident id, `payload` held a bare string instead of a dict, and `max_attempts` **held the error message.** Every dead-letter entry the pipeline produced was malformed — the failure evidence a SOC product exists to preserve.

Class: positional coupling — the *first* defect this council ever reported, reintroduced by the author who was fixing it elsewhere. Fixed with keyword arguments and an AST test that rejects any positional `dlq.enqueue` call.

F7 — Unfenced leases and non-atomic acceptance *(High, partially open)*

Two seats independently: `SKIP LOCKED` proves mutually exclusive claiming **at one instant only**. `complete()` and `fail()` do not check `claimed_by`, so a worker whose lease expired can overwrite the state of the worker that legitimately reclaimed the item. Delivery is at-least-once, not exactly-once, and the V4 documentation was not careful about that distinction.

Separately, the incident row and the queue insert are not written in one transaction, so the API can answer `"processing": true` before durable work exists.

Status: OPEN. Both are real; neither is fixed. Fencing tokens and a transactional outbox are the top two items of the 90-day plan.

Part II — What genuinely held

The council confirmed these against running code, and two of three seats had every incentive to find holes in them.

- **The `incident_state` split-brain is genuinely gone.** Claude: *"the `incident_state` split-brain is genuinely gone (`repository.py:178-218`)"*. Codex: *"Lifecycle split-brain is eliminated."*
- **Redis deletion was judged correct**, including the reasoning that a second stateful store would recreate the split-authority class.
- **Startup validation** refusing SQLite with `replicas > 1` was marked ELIMINATED by Antigravity.
- **Phantom execution remains impossible by type**, unchanged since v3.
- **The English detection result is real for this benchmark.** Codex verified the confusion matrix independently: 282 TP, 48 FN, 8 FP, 174 TN.

Part III — Where the seats disagreed, and what they could not verify

No seat could independently confirm the live-PostgreSQL claim. All three hit the same wall: pytest's temporary-directory creation is denied in their sandboxes. Claude states it plainly — *"nobody has independently confirmed a live-Postgres pass."* The 23 live tests pass when run outside the seats' environment, but that is the author's assertion, not the council's finding, and it is recorded as such.

On the detection number, the seats split on severity rather than fact. All three accept the confusion matrix and all three reject the headline. Codex is most precise: attack-only sources (Gandalf, MultiJail) and benign-only

sources (xquad, synthetic) leave **provenance correlations**, so the classifier may be learning which dataset a string came from. The prescribed test — leave-one-source-out — has not been run.

On FPR, Codex did the arithmetic: at 100,000 benign events/day, FPR 0.044 is **~4,400 false alerts/day**, roughly 37 analyst-hours even at 30 seconds each. Claude's range is 440–4,400. Neither treats 0.855 recall as compensating.

Part IV — What has been fixed since the vote

#	Finding	Class	Fix	Tests
F1	Retention crashed silently	silent scheduled failure	Targets live tables; jobs log, count, expose `healthy`, and gain `run_once()` that raises	4
F2	Second metrics store	split authority	`app/metrics/` deleted; AST test rejects any file-persisted counter	3
F3	SQLite deadlock	lock held across callback	`elected()` records, releases, *then* yields; Postgres uses a dedicated lock connection	3
F4	`INSERT OR IGNORE`	semantics lost in translation	Real `ON CONFLICT DO NOTHING`; the bug-pinning v3 test corrected	3
F5	Retention broke the invariant	invariant not maintained under delete	Incident-scoped deletion: an incident and its artifacts share a fate, in one transaction	3
F6	DLQ transposition	positional coupling	Keyword arguments; AST test forbids positional `dlq.enqueue`	3
F7	Unfenced leases, non-atomic accept	—	**NOT FIXED** — top of the 90-day plan	—

Test suite: 220 standard + 23 live = 243 passing, up from 222 at the time of the vote.

Three of these fixes exposed further problems in the *tests* rather than the code — a test that pinned the F4 bug as correct, a test that encoded pre-fix retention semantics, and a counter-detection test that matched `json.dumps` (serialising a database column) as well as `json.dump` (writing a file). That last one is the third time in this project an over-broad substring match has produced a false positive.

Part V — The 90-day plan

Merged from all three seats; they converged closely.

1. **Atomic acceptance and fenced leases.** Write the incident and its queue item in one transaction; return `"processing": true` only after commit. Add an ownership predicate to `complete`/`fail` and treat zero rows affected as lost ownership. *(F7, open. ~1 week.)*

2. **Resumable pipeline.** `process()` must resume from stored lifecycle state rather than restarting at `VALIDATED`, and the DLQ needs a real drain scheduler. *(~3 days.)*
3. **Concurrency-safe evidence ledger.** Lock the per-tenant ledger head and append within one transaction. Until that passes, stop describing the ledger as non-repudiable. *(~2 weeks.)*
4. **Honest detection evaluation.** Leave-one-source-out scoring, and benign/attack drawn from the *same* corpora. Publish cross-source recall, not the confounded 0.855. *(~1 week.)*
5. **One instrumented design-partner deployment.** Observe mode, human approval intact, against a real IdP, collector, PostgreSQL cluster and threat feed, with acceptance SLOs defined in advance. *(4–6 weeks.)*

Part VI — What is still not true

Not implemented: SAML, SCIM, Kafka, in-app mTLS — each out of scope with a documented reason.

Implemented but never driven by live infrastructure: OIDC (no identity provider), OpenTelemetry (no collector), Prometheus (never scraped), Helm and Kubernetes (never applied; `helm` is not installed, so the chart is not even lint-verified).

Not validated: multilingual detection, enforced as unclaimable by a CI test. The ECDSA recovery primitive remains unaudited. No production traffic has ever been processed.

Open defects: F7 in full — unfenced leases and non-atomic acceptance.

A credential still requires rotation by the user. A key of the form `sk-synth-…` remains inside `sentinel_soc-main.zip` in the Downloads folder. It must be rotated at the issuing service.

Part VII — Honest bottom line

Four reviews, four unanimous rejections, four confirmations that failure was relocated rather than eliminated.

It would be easy to read that as no progress. It is not: PostgreSQL genuinely works, the queue genuinely survives restarts, dead infrastructure is genuinely gone, and the detection number is now measured against data nobody here wrote. The defects found this round are real but narrower and deeper than the ones found in v1 — the system has moved from "does not work" to "works, with specific unsafe boundaries."

What has *not* improved is the thing that matters. Each pass fixes the instances it is shown and ships new instances of the same classes. This round produced the clearest evidence yet of why: **F1's fix activated F3.** A repair enabled a latent deadlock, because the repair and the latent bug were in different files reviewed at different times, and nothing in the system understood that a job which starts working will start acquiring locks.

The classes that keep recurring — positional coupling, split authority, invariants unenforced on one path, silent failure of scheduled work — are not carelessness. They are what happens when correctness is maintained by a person's attention rather than by structure. Every fix in Part IV that will actually hold is the one that made the mistake *unrepresentable*: keyword-only arguments, an AST test, one schema, incident-scoped deletion. The ones that will not hold are the ones that depend on the next author remembering.

Commercially, nothing has changed and the seats were unanimous about why. FPR 0.044 means thousands of false alerts a day at realistic volume. The detection number rides an admitted provenance confound. No live identity provider, cluster, or customer has ever run this. It remains a competent prototype of a control plane, not a product — and, as Claude's closing puts it, the code shows no sign of pursuing the one opening the market

research actually identified.

Appendix A — Verification method

Every finding in Part I was reproduced against running code before publication. F3 was confirmed by a probe that hung for its full 30-second timeout; F5 by driving an incident to `COMPLETED` and ageing its artifacts; F4 by translating a statement and inspecting the output.

Fixes were verified by executing the failing case, applying the change, and re-executing. Where a claim could not be reproduced as stated, it is reported as such: the first form of F5 tested — deleting artifacts of an incident whose state row also aged out — turned out to be *safe*, and only the asymmetric case was a defect. Reporting the narrower true version rather than the broader claimed one is the point.

Two failures in this review's own process are recorded rather than hidden: the run that reused v3 transcripts, and a third instance of an over-broad substring match producing a false positive.

Appendix B — Artifacts

Artifact	Path
Nine round transcripts, verbatim	`council/transcript/`
Run metadata and timings	`council/transcript/run_summary.json`
Research brief given to the seats	`council/brief.md`
Regression tests for all V4 findings	`tests/test_council_v4_regressions.py`
Enterprise readiness matrix	`docs/READINESS.md`
Architecture decisions and exclusions	`docs/ARCHITECTURE_DECISIONS.md`
Benchmark methodology and confound	`evaluation/BENCHMARK.md`
Previous reviews	`council/COUNCIL_REPORT_v1.*`, `_v2.md`, `_v3.*`
Previous transcripts	`council/transcript_v1_prefix/`, `_v2_archive/`, `_v3_archive/`